

Сайт, которым *пользуются*.

До этого мы строили статические сайты — красивые, но без «своих» данных у каждого пользователя. Сегодня учим Claude собирать сервер, базу данных, регистрацию — и запускать всё это в интернет, чтобы реальные люди могли пользоваться.

ДЛИТЕЛЬНОСТЬ

8–10 дней · 2–4 ч/день

ЧТО ВЫ ОСВОИТЕ

frontend + backend + БД · авторизация · деплой · Claude API

ГЛАВНЫЙ ИТОГ

сайт с живой ссылкой и регистрацией

АРТЕФАКТ

личный инвест-кабинет и AI-консультант на Claude API в интернете

Этап интенсивный по инструментам: нужны два аккаунта, ключ Anthropic API и один skill. Все шаги — одной командой и кликом, ничего сложного.

ОБЯЗАТЕЛЬНО · АККАУНТ

Vercel

Что это: платформа для деплойа frontend-приложений в один клик. Подключаете свой GitHub-репозиторий — Vercel сам обновляет сайт после каждого `git push`.

Зачем сейчас: на нём публикуем frontend нашего инвест-кабинета (он сложнее статических HTML, поэтому GitHub Pages недостаточно).

Где взять: vercel.com/signup. Войдите через свой GitHub-аккаунт — всё связывается автоматически. Бесплатный тариф щедрый: до 100 ГБ трафика в месяц.

Проверить: можете войти на vercel.com и увидеть свой dashboard.

ОБЯЗАТЕЛЬНО · АККАУНТ

Railway

Что это: платформа для деплоя *backend*-частей: серверов, баз данных, фоновых задач. То, что не умеет Vercel.

Зачем сейчас: на нём запускаем сервер инвест-кабинета и PostgreSQL-базу — всё в одном месте.

Где взять: railway.app. Регистрация через GitHub. Бесплатный тариф: \$5 кредитов в месяц — для учебного проекта более чем достаточно.

Проверить: dashboard открывается, в нём пусто.

ОБЯЗАТЕЛЬНО · API-КЛЮЧ

Anthropic API key

Что это: ключ от Claude API — того же Claude, но вызываемого из программ, не только из Claude Code. Через него мы встраиваем AI прямо в наше приложение.

Зачем сейчас: для второго проекта — AI-консультанта по финансам.

Где взять: console.anthropic.com/settings/keys. Регистрация бесплатная, при создании ключа получите \$5 кредитов на старт.

Куда положить: в `.env` в корне проекта (`ANTHROPIC_API_KEY=sk-ant-...`)

и **ОБЯЗАТЕЛЬНО** в `.gitignore`.

На Vercel/Railway: ключ загружается через интерфейс «Environment Variables», а не через файл – разберём в § I.

ВСТРОЕНО • SKILL

claude-api

Что это: skill от Anthropic, который учит Claude правильно интегрировать Claude API в ваши приложения – с prompt caching, выбором модели, обработкой ошибок.

Зачем сейчас: чтобы AI-консультант работал с первого раза, а не разваливался на границах.

Устанавливать не нужно: `claude-api` – *встроенный* skill, идёт вместе с самим Claude Code.

Проверка: в `claude` напечатайте `/skills`, `claude-api` должен быть в списке.

Откуда берутся skills. К этому моменту вы встретили три источника: (1) *встроенные* – уже внутри `claude` (как сегодняшний `claude-api`); (2) *Smithery* – внешний реестр, ставится через `npx @smithery/cli@latest skill add ...` (как `frontend-design` в этапе 02); (3) *встроенный маркетплейс Claude Code* (`claude-plugins-official`) – ставится slash-командой `/plugin install ...@claude-plugins-official` (как `superpowers` с этапа 01). Один skill может жить и в *Smithery*, и в маркетплейсе одновременно – это нормально.

ЖЕЛАТЕЛЬНО • IDE

Расширение Claude Code для VS Code

Что это: официальное расширение, которое встраивает Claude Code прямо в VS Code. Видно изменения в файлах в реальном времени, можно общаться с Claude в боковой панели, не переключаясь в терминал.

Зачем сейчас: в большом проекте удобно видеть код и диалог одновременно.

Где взять: в VS Code: вкладка Extensions, поиск «Claude Code», установить.

Проверить: в боковой панели появляется иконка Claude.

Frontend, backend и БД — «зал, кухня и кладовая»

Метафора, которую полезно держать в голове весь этап — ресторан:

- **Frontend** = **зал**. Что видит и щупает посетитель: меню, столики, оформление. В нашем случае — страницы, кнопки, формы, графики. Живёт в браузере пользователя.
- **Backend** = **кухня**. Готовит блюда. Принимает заказы из зала, лезет на склад, отдаёт еду. В нашем случае — сервер, который принимает запросы со страниц, читает базу, считает, отправляет результат.
- **База** = **кладовая**. Тут лежат продукты. В нашем случае — пользователи, их операции, настройки. Кладовую видит только кухня, гости туда не ходят.

Эта метафора важна, потому что:

- Frontend и backend — это *две разные программы*. Они часто живут в двух разных папках, на двух разных серверах, и общаются между собой через API.
- Секреты (ключи, пароли) живут *только в backend*. Frontend их не видит — иначе любой гость может украсть их через DevTools.
- База — *отдельный сервис*, к которому подключается backend. Управлять ей можно отдельно: бэкапить, мигрировать, переключать.

Backend на FastAPI: «кухня» на Python

Для backend в курсе используем **FastAPI** — современный python-фреймворк, который Claude хорошо знает и собирает за минуты. Минимум кода — и у вас работающий сервер с документацией.

Сервер на FastAPI — это набор «эндпоинтов». Пример:

```
# читать список операций
GET /api/operations      → список JSON

# добавить операцию
POST /api/operations     → принимает JSON, возвращает созданное

# войти в аккаунт
POST /api/login          → принимает email + password, возвращает токен
```

Каждый эндпоинт – это функция в Python, которую пишет Claude. Вы её не трогаете, но понимаете *что она делает*.

FastAPI бесплатно даёт автоматическую документацию по `/docs` – вы можете тыкать кнопками и тестировать сервер без написания клиента.

База данных: SQLite в разработке, PostgreSQL в интернете

SQLite – это «база в файле». Один файл `app.db` – и там вся ваша информация. Идеальна для разработки и учебных проектов: нечего настраивать, всё работает.

PostgreSQL – это «настоящая» база. Отдельный процесс, к которому подключается ваш backend. Поддерживает множество пользователей одновременно. На Railway она ставится в один клик.

Стратегия в курсе:

- На вашем компьютере (разработка) – SQLite. Просто файл.
- В интернете (Railway) – PostgreSQL.
- Между ними переключаемся одной строкой в `.env`:

`DATABASE_URL=sqlite:///./app.db` локально

и `DATABASE_URL=postgres://...` на Railway. Код один и тот же – меняется только переменная окружения.

Базой управляет ORM (например, SQLAlchemy) – библиотека, которая превращает Python-объекты в запросы SQL. Вам не нужно знать SQL: вы говорите Claude «создай модель User с email, password_hash, created_at», и он делает всё остальное.

Авторизация: регистрация, вход, защита

Чтобы у каждого пользователя были свои данные, нужна авторизация. Минимум:

1. **Регистрация:** email + пароль → создаётся запись в таблице `users`. Пароль *хешируется* (превращается в нечитаемую строку) – в базе никогда не лежит «как есть».
2. **Вход:** email + пароль → сервер проверяет хеш, выдаёт JWT-токен.
3. **Защита эндпоинтов:** когда браузер делает запрос к `/api/operations`, он шлёт токен. Сервер декодирует токен, узнаёт, какой это пользователь, и возвращает *только его* данные.
4. **Выход:** браузер просто стирает токен у себя.

Что Claude делает за вас:

- пишет регистрацию и вход;
- хеширует пароли через `bcrypt` или `argon2`;
- выпускает и проверяет JWT;
- защищает эндпоинты «декоратором» `require_auth`;
- добавляет «забыл пароль», 2FA, OAuth – если попросите.

Что важно понимать вам:

- пароли в базе – *только в виде хеша*. Если кто-то украдёт базу, ваши пароли всё равно не видны;
- JWT-секрет (ключ для подписи токенов) живёт *только в backend*, в `.env`;
- HTTPS обязателен для любых страниц с вводом пароля – и Vercel/Railway включают его автоматически.

Деплой одной кнопкой: Vercel + Railway

«Деплой» больше не значит «арендовать сервер, настраивать nginx, ставить SSL». Современный подход:

1. Связали аккаунт **Vercel** с вашим GitHub.
2. В Vercel «Import Project» – выбираете репозиторий с frontend.
3. Vercel сам понимает, что это Next.js (или React, или Vue), собирает и публикует. Через 30 секунд – живая ссылка `my-app.vercel.app`.
4. Аналогично в **Railway**: «New Project» → ваш backend-репозиторий + добавить «PostgreSQL» сервис.
5. В настройках добавляете environment variables (например, `ANTHROPIC_API_KEY`, `DATABASE_URL`).
6. Каждый `git push` – новый деплой. На каждый PR – своя preview-ссылка для ревью.

Что важно:

- **Никогда не кладите секреты в код.** Только Environment Variables через интерфейс платформы.
- **Настройте CORS** – чтобы ваш Vercel-frontend имел право обращаться к Railway-backend. Claude настроит, но знать про это надо.
- **Бесплатные тарифы засыпают.** Если приложение не использовали несколько часов, первый запрос будет долгий (контейнер просыпается). Это нормально для учебных проектов.

Claude API в своём приложении: модели и prompt caching

Три модели, между которыми выбираем для разных задач:

- **Opus** – самая умная, самая дорогая. Для сложных аналитических задач, где качество важнее всего (например, финансовая аналитика капстоуна).
- **Sonnet** – «золотая середина». 80% качества Opus за 20% денег. Большинство приложений – на ней.
- **Haiku** – быстрая и дешёвая. Для простых классификаций («это что – еда или транспорт?») и высоких объёмов.

В коде это одна строка:

```
model="claude-sonnet-4-6"    # для типичных задач
model="claude-opus-4-7"     # для сложного анализа
model="claude-haiku-4-5"    # для классификации
```

Prompt caching экономит деньги и время:

- Если в каждом запросе вы шлёте один и тот же «контекст» (например, описание AI-консультанта, схему ваших данных, last 30 операций пользователя) – Claude API *кэширует* эту часть и на повторных запросах берёт за неё в 10 раз меньше.
- Кэш живёт 5 минут. Если запросы идут чаще – почти всегда попадаете в кэш.
- Для нашего AI-консультанта это критично: пользователь шлёт по 15 вопросов подряд, кэшируется системный промпт + его портфель.

Skill `claude-api` учит Claude правильно ставить точки кэширования.

Без skill ваш AI-консультант будет работать в 5–10 раз дороже.

Что у Claude получается «не сразу» — на что обращать внимание

На этом этапе впервые «настоящая» сложность. Не всё работает с первого раза. Что чаще всего ломается:

- **CORS-ошибки.** Frontend на Vercel не может позвонить Railway-backend, потому что серверы разные. Claude всё настроит, но первый раз скорее всего получите ошибку «blocked by CORS policy». Решение — «добавь *cors middleware с allowed origins* из *.env*».
- **Секреты не подгрузились.** Локально работает, на Vercel/Railway — нет. Проверьте, что переменные окружения добавлены в dashboard платформы (не только в `.env` на вашем компьютере).
- **Миграции базы.** При первом деплое таблиц ещё нет. Claude обычно делает это автоматически, но иногда нужно явно: «добавь *startup-хук, который создаёт таблицы при старте сервера*».
- **Спящий бэкэнд.** Первый запрос после паузы — долгий. Это нормально на бесплатных тарифах. Решение для демо — пинговать backend каждые 5 минут (например, через cron-задачу — этап 10).

Дисциплина: на каждой проблеме смотрите Claude'у в браузерные DevTools (через Playwright) и логи backend. Большинство решается за 5–15 минут.

PRACTICUM

Прогон · минимальный full-stack за 30 минут

1. Создайте папку `hello-fullstack`, две подпапки: `backend/` и `frontend/`. В корне — `CLAUDE.md` и `.gitignore`.
2. Войдите в Plan Mode и попросите Claude:

```
> составь spec.md для минимального full-stack «hello-fullstack»:  
> — backend: FastAPI с одним эндпоинтом GET /api/hello,  
>   возвращает {"message": "Hello, <ваше имя>"} с timestamp.  
>   локально: uvicorn на 8000.  
> — frontend: одна html-страница, fetch на /api/hello,  
>   рендерит сообщение в крупный заголовок.  
> — оба готовы к деплою на Railway (backend) и Vercel (frontend).  
> — CORS настроен для localhost и для будущих vercel.app.  
> критерии готовности:  
>   1) backend стартует одной командой;  
>   2) frontend открывается в браузере;  
>   3) кнопка «обновить» получает новый timestamp.
```

3. Утвердите spec, дайте исполнять.
4. Локально проверьте: backend запущен на `:8000`, frontend — через `python -m http.server 5000`. Должно работать.
5. Через playwright — ревизия:

```
> через playwright открой http://localhost:5000.  
> нажми «обновить» три раза. timestamp каждый раз меняется?  
> есть ли ошибки в console?
```

6. Деплой:

```
> запущь backend на github как hello-fullstack-backend (публичный).  
> деплой на Railway: новый проект из этого репо.  
> запущь frontend как hello-fullstack-frontend.  
> деплой на Vercel: импорт из репо.  
> в обеих платформах загрузи нужные env vars.  
> замени в frontend URL backend-а на railway-ссылку.  
> запущь обновление.
```

7. Откройте Vercel-ссылку. Убедитесь, что работает по живому URL.

8. Если что-то не работает: смотрите логи Railway и DevTools во Vercel-странице. Сообщите Claude точное сообщение об ошибке – обычно решает за 1–2 итерации.

Что мы тренируем: **минимальный, но настоящий full-stack – от пустой папки до живой ссылки**. Дальше – на этом фундаменте делаем серьёзные приложения.

ПРОЕКТ А · ПО ИНСТРУКЦИИ

Личный инвестиционный кабинет

Цель: полноценное приложение с регистрацией, базой и деплоем. Каждый пользователь видит свой портфель и свои прогнозы. URL вида `https://my-investing.vercel.app`, который не стыдно показать.

Фазы (через spec → Plan Mode → TodoWrite, как в этапе 04)

- 1. Спеc и скелет.** Папки `backend/` и `frontend/`, `CLAUDE.md`, `.gitignore`, `spec.md` с разделами «модели данных» (User, Operation, Forecast), «эндпоинты», «UX-флоу».
- 2. Backend: модели и база.** FastAPI + SQLAlchemy + SQLite (локально). Модели: User, Operation, Forecast. Миграции через Alembic либо «создать таблицы при старте».
- 3. Backend: авторизация.** Регистрация, вход, JWT. Эндпоинты `/api/auth/signup`, `/api/auth/login`, `/api/auth/me`. Хеширование bcrypt. Декоратор `require_auth`.
- 4. Backend: операции.** CRUD `/api/operations` с фильтром «только мои». Импорт CSV через `/api/operations/import` (multipart upload).
- 5. Backend: прогноз.** Эндпоинт `/api/forecast`: входы (как в этапе 04), выходы – три сценария.
- 6. Frontend: каркас.** Next.js или просто React-страницы. Маршруты: `/` (лендинг), `/signup`, `/login`, `/dashboard`, `/forecast`.
- 7. Frontend: формы и дашборд.** Используем skill `frontend-design` для вёрстки. Внутри – графики из этапов 02–04.
- 8. Frontend: интеграция с backend.** Хранение JWT в `localStorage`. Авто-добавление токена в каждый запрос.
- 9. Деплой backend на Railway.** Создание PostgreSQL-сервиса. Миграция SQLite → PostgreSQL через переменную `DATABASE_URL`.
- 10. Деплой frontend на Vercel.** Подмена URL backend на production. CORS настроен.
- 11. Verification.** Через playwright: регистрация, вход, импорт CSV, прогноз. Сверка чисел с этапом 02 (на тех же данных).

Что вы здесь освоили

- двух-репозиторийный full-stack: backend и frontend как отдельные проекты;
- работу с двумя БД одновременно (SQLite локально, PostgreSQL в production);
- авторизацию пользователей с хешированием паролей и JWT;

- деплой через Vercel + Railway, переменные окружения, CORS;
- сборку приложения, которым реально могут пользоваться люди.

AI-консультант по финансам на Claude API

Цель: добавить в инвест-кабинет вкладку «AI-консультант». Чат, который видит ваши данные (через *backend*), отвечает на вопросы по портфелю, использует *prompt caching* и разные модели под разные задачи.

Шаги

1. В `backend/` добавьте эндпоинт `/api/advisor/ask`:

```
> используя skill claude-api,  
> добавь эндпоинт POST /api/advisor/ask.  
> принимает: { question: string }.  
> собирает контекст: системный промт «ты — финансовый консультант» +  
> портфель текущего пользователя (через require_auth) + последние 30 операций.  
> вызывает claude-sonnet-4-6 с prompt caching на системный промт + портфель.  
> возвращает { answer: string, model: string, tokens_used: int }.  
> ВАЖНО: ANTHROPIC_API_KEY — только из env, никогда не светится в логах.
```

2. В `frontend/` добавьте страницу `/advisor` с чатом. История сообщений в state, ответы streaming-ом.

3. Уточните UI:

- «сделай вид «классической переписки», узкая колонка, тёплая палитра»;
- «в каждом ответе показывай маленькую плашку «модель + tokens», для прозрачности»;
- «добавь три быстрых вопроса-кнопки сверху: «оцени мой риск», «куда переложить?», «прогноз на год»».

4. Обработка ошибок:

```
> добавь обработку:  
> — если ANTHROPIC_API_KEY не задан — friendly ошибка «AI выключен».  
> — если кончились кредиты Anthropic — друг. ошибка «лимит, попробуйте позже».  
> — если backend не отвечает — повтор через 3 секунды, потом ошибка.
```

5. Multi-model логика:

```
> добавь второй эндпоинт /api/advisor/classify:  
> принимает «что эта операция?», использует haiku  
> (классификация транзакций — простая задача, экономим деньги).  
> покажи в advisor.html, как переключаются модели.
```

6. Деплой backend на Railway, frontend на Vercel. В Railway добавьте ANTHROPIC_API_KEY через интерфейс (НЕ коммитьте в .env).

7. Verification:

```
> через playwright прогон:
> – войди как тестовый user;
> – открой /advisor;
> – задай 3 вопроса по портфелю;
> – проверь что:
>   1) ответы релевантны (упоминают тикеры из моего портфеля);
>   2) tokens_used меньше у второго и третьего вопроса (cache hit);
>   3) classify-эндпоинт использует haiku, а ask – sonnet.
> пришли отчёт.
```

8. Финальный аудит: «если кто-то получит мой Anthropic API key, как он узнает, через какое моё приложение он утёк? есть ли в логах что-то, что я должен очистить?»

Что вы здесь освоили

- интеграцию Claude API в своё приложение через skill `claude-api` ;
- prompt caching для экономии денег и ускорения чата;
- выбор модели под задачу (Sonnet для аналитики, Haiku для классификации);
- обработку ошибок API и graceful fallback;
- безопасность: ключи только в env, не в логах.

ПРОЕКТ С · САМОСТОЯТЕЛЬНО

Сервис коллекции с регистрацией

Цель: вариация проекта А – тот же full-stack стек (FastAPI + PostgreSQL + Vercel-frontend + JWT-auth), но для вашей коллекции: книг, настольных игр, кофеен, мест города. Каждый пользователь ведёт свою.

Подсказки

- Тот же скелет, что в А. Меняется только модель данных и UX.
- Модель: User + Item + Tag. У Item: title, description, image_url, rating, tags.
- UX: добавить элемент с загрузкой картинки, поиск по коллекции, фильтры по тегам/рейтингу, страница «случайный элемент».
- Картинки можно хранить через сторонний сервис (например, Cloudinary бесплатно).
- Используйте опыт этапа 05 для двух репозиторийев.

РАСКРЫТЬ ПОСЛЕ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

– сначала сделайте, потом проверяйте

ГОТОВО, ЕСЛИ...

- ✓ есть две публичные ссылки: backend на Railway и frontend на Vercel;
- ✓ можно зарегистрироваться, войти, добавить минимум 5 элементов;
- ✓ другой человек регистрируется и видит *только свою* коллекцию (проверьте – зарегистрируйтесь дважды разным email);
- ✓ работает поиск или фильтры;
- ✓ в логах backend нет API-ключей и паролей (проверьте через Railway dashboard);
- ✓ через playwright прогнан полный сценарий «новый пользователь → регистрация → 3 элемента → выход → вход».

СПРОСИТЕ CLAUDE НАПОСЛЕДОК

«Прогуляйся по моему сервису через playwright как security-эксперт. Найди три места, где я мог бы случайно «утечь» данные одного пользователя другому. Как это исправить?»

Telegram-бот для вашего инвест-кабинета

Цель: бот, который ходит в ваш *backend* (из проекта A) и даёт быстрый доступ к функциям с телефона. Команды: `/login`, `/portfolio`, `/last`, `/forecast`, `/ask`.

Подсказки

- Бот – отдельный python-процесс на Railway. Использует библиотеку `aiogram` или `python-telegram-bot`.
- Создайте бота через @BotFather, получите токен. Положите в Railway env vars.
- Авторизация по Telegram: бот сохраняет связь `telegram_id` → `user_id` в базе. После `/login email password` – токен JWT хранится у бота на время.
- Команды бот делает запросами к тому же backend, что использует frontend.
- Команда `/ask` – передача в AI-консультант (проект B).
- Тестовый сценарий: после `/login` и `/last` – бот отвечает 3 последними операциями.

РАСКРЫТЬ ПОСЛЕ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

– сначала сделайте, потом проверяйте

ГОТОВО, ЕСЛИ...

- ✓ бот живёт в Railway как отдельный сервис, токен в env, не в коде;
- ✓ после `/login` и `/portfolio` бот возвращает текущее состояние портфеля (как в вебе);
- ✓ `/last` возвращает 3 последние операции;
- ✓ `/forecast` вызывает реальный эндпоинт прогноза;
- ✓ `/ask что-то` идёт в AI-консультанта и возвращает ответ;
- ✓ другой telegram-аккаунт после `/login` другим email видит *только свой* портфель;
- ✓ в логах нет ни токенов, ни паролей.

СПРОСИТЕ CLAUDE НАПОСЛЕДОК

«Если кто-то скомпрометирует мой Telegram-токен, что он сможет сделать с моими пользователями? Что нужно изменить, чтобы при компрометации токена бот «умер», а не стал атакером?»

Что добавилось к словарю

Frontend	«Зал ресторана» — часть приложения, которая работает в браузере пользователя. То, что человек видит, кликает, заполняет.
Backend	«Кухня» — программа на сервере, принимающая запросы от frontend, обращающаяся к базе и возвращающая данные. Видит секреты и логику. Пользователь его не видит.
База данных	«Кладовая» — постоянное хранилище данных приложения: пользователи, их операции, настройки. Доступна только backend.
SQLite	База данных «в одном файле». Идеальна для разработки. Один файл app.db — вся ваша информация.
PostgreSQL	Полноценная база как отдельный сервис. Для production. На Railway ставится в один клик.
FastAPI	Современный python-фреймворк для backend. Минимум кода, бесплатная авто-документация по /docs. Используем как «кухню».
Эндпоинт	Одна функция backend, доступная по URL. Например, POST /api/login — «эндпоинт логина». Имеет HTTP-метод и формат данных.
JWT-токен	JSON Web Token — короткая строка, которая «доказывает» серверу, что вы вошли. Хранится в браузере, шлётся на каждый запрос. Подписан секретом backend.
CORS	Cross-Origin Resource Sharing — защита браузера: страница с одного домена по умолчанию не может звонить серверу с другого. Настраивается в backend, чтобы Vercel-frontend мог обращаться к Railway-backend.
Деплой	Процесс «отправить ваше приложение на сервер в интернете». Современный подход — в один git push через Vercel или Railway.
Vercel	Платформа для деплоя frontend. Подключаете GitHub-репо — каждый push автоматически публикует новую версию. Бесплатный тариф щедрый.
Railway	Платформа для деплоя backend и баз. Один сервис — и backend, и PostgreSQL рядом. \$5 кредитов в месяц бесплатно.
Claude API	Программный интерфейс к моделям Claude от Anthropic. Через него ваше приложение само вызывает Claude (а не только вы через Claude Code). Ключ в ANTHROPIC_API_KEY.
Prompt caching	Кэширование одинаковых частей промпта между запросами. Если в начале каждого запроса одно и то же, Claude API кэширует и берёт за это в 10 раз меньше денег и времени. Кэш живёт 5 минут.
Модели Claude (Opus	Три модели разной мощности и цены. Opus — самая умная, для сложного

01 Frontend, backend, БД — в метафоре «ресторан» что чему соответствует?

Frontend = зал (что видит и щупает гость). Backend = кухня (готовит блюда, лезет на склад). БД = кладовая (там лежат продукты, видит её только кухня).

02 Почему пароли в базе хранятся хешированными, а не «как есть»?

Если кто-то украдёт базу — ваши пароли всё равно не видны. Хеш — необратимая «мясорубка»: из пароля считается строка, а из строки пароль обратно не вытащить. При входе сравнивается хеш введённого пароля с хешем в базе.

03 Что такое CORS-ошибка и где её чинят?

Браузер блокирует запросы от страницы на одном домене (Vercel) к серверу на другом (Railway), если backend явно не разрешил это. Чинится в backend: добавляется CORS middleware с разрешённым origin (вашим Vercel-доменом).

04 Почему `ANTHROPIC_API_KEY` *никогда* нельзя класть во `frontend`?

Frontend живёт в браузере пользователя. Любой откроет DevTools и увидит ваш ключ. Через минуту его украдут — и будут жечь ваши деньги через ваш аккаунт. Ключ должен быть только в backend, в env vars Railway. Frontend обращается к вашему backend-у, backend — к Anthropic.

05 Какую модель **Claude** взять для «классификация транзакции» и какую — для «годовой финансовый прогноз с аргументацией»?

Классификация транзакций — **Haiku** (быстрая, дешёвая, простая задача). Годовой прогноз с аргументацией — **Opus** (сложно, нужна качественная аргументация). Большинство «средних» задач — на **Sonnet**.

06 Зачем **prompt caching** в AI-консультанте?

В каждом запросе мы шлём один и тот же системный промпт + портфель пользователя (десятки строк). Без кэша Claude API обрабатывает их каждый раз заново – долго и дорого. С кэшем эти части обрабатываются один раз, дальше используются «как есть» – в 10 раз быстрее и дешевле. Особенно важно, когда пользователь шлёт серию вопросов подряд.

07 Что лучше для разработки и что для **production**: SQLite или PostgreSQL?

SQLite – для разработки и учебных проектов: один файл, ничего настраивать. PostgreSQL – для production, особенно когда несколько пользователей одновременно. Хорошая практика – код один и тот же, разница только в `DATABASE_URL` в `.env`: `sqlite:///./app.db` локально, `postgres://...` на Railway.

Что я теперь умею

- ☐ Понимаю архитектуру full-stack: frontend, backend, БД и их ответственности.
- ☐ Знаю, как Claude собирает FastAPI-сервер с эндпоинтами и БД.
- ☐ Понимаю авторизацию: хеш паролей, JWT, защита эндпоинтов.
- ☐ Опубликовал backend на Railway и frontend на Vercel, с env vars.
- ☐ Встроил Claude API в своё приложение через skill `claude-api`.
- ☐ Выбираю модель Claude (Opus / Sonnet / Haiku) под задачу и применяю prompt caching.
- ☐ Сделал четыре проекта: инвест-кабинет, AI-консультант, сервис коллекции, Telegram-бот.

Готовы закрыть этап? Нажатие фиксирует прогресс на главной странице.

«Сайт без регистрации — красивая
открытка. Сайт с аккаунтами и живыми
данными — продукт».

Этап 06 пройден — вы умеете собирать настоящие
приложения. Дальше — самый интересный поворот:
учим Claude нанимать собственную команду стажёров.

ВНУТРИ ЭТАПА

- § 0 · Подготовка
- § I · Темы
- § III · Проекты
- § V · Словарь
- § VI · Quiz
- § VII · Чек-лист

КУРС

- К оглавлению
- ← Этап 05
- Этап 07